

Unit 3: Lists, Tuples, Dictionaries, Sets, and Strings

3.1 Introduction to Lists

A **list** is a built-in data structure in Python used to store multiple items in a single variable.

Lists are:

- **Ordered** (items have a defined order),
- **Mutable** (can be changed after creation),
- **Heterogeneous** (can hold different data types).

Example:

```
fruits = ["apple", "banana", "cherry"]
numbers = [10, 20, 30, 40]
mixed = [1, "hello", 3.5, True]
```

List Slicing

Allows you to access parts of a list using index ranges:

```
fruits[1:3]    # ['banana', 'cherry']
fruits[:2]     # ['apple', 'banana']
fruits[-1]     # 'cherry' (last element)
```

in Operator

The **in** operator is used to **check if an element exists in a list** (or other iterable types like strings, tuples, sets).

- It returns True if the element is found.
- Returns False if the element is not present.

Example:

```
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits)  # Output: True
print("orange" in fruits) # Output: False
```

list methods:

append()

Adds an item to the **end** of the list.

```
fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits) # ['apple', 'banana', 'cherry']
```

index()

Returns the **index** of the first occurrence of the given item.

```
fruits = ["apple", "banana", "cherry"]
print(fruits.index("banana")) # 1
```

insert()

Inserts an item at a **specific position**.

```
fruits = ["apple", "cherry"]
fruits.insert(1, "banana")
print(fruits) # ['apple', 'banana', 'cherry']
```

sort()

Sorts the list **in-place** in ascending order. For descending order, use `reverse=True`.

```
numbers = [3, 1, 4, 2]
numbers.sort()
print(numbers) # [1, 2, 3, 4]
```

```
numbers.sort(reverse=True)
print(numbers) # [4, 3, 2, 1]
```

remove()

Removes the **first matching item**.

```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")
print(fruits) # ['apple', 'cherry', 'banana']
```

reverse()

Reverses the **order** of the list.

```
fruits = ["apple", "banana", "cherry"]
fruits.reverse()
print(fruits) # ['cherry', 'banana', 'apple']
```

min() and max()

Used to find the **smallest** and **largest** items in a list.

```
numbers = [5, 2, 9, 1]
print(min(numbers)) # 1
print(max(numbers)) # 9
```

3.2 List Comprehension

List Comprehension is a concise and elegant way to create new lists by applying an expression to each item in an existing iterable (like a list, range, etc.).

Basic Syntax:

```
new_list = [expression for item in iterable if condition]
```

- **expression:** The value to add to the new list.
- **iterable:** A sequence (like list or range) to loop through.

- **condition (optional):** Only include the item if this is True.

Examples:

◆ *Square of numbers from 0 to 4*

```
squares = [x**2 for x in range(5)]
print(squares) # [0, 1, 4, 9, 16]
```

3.3 Two-Dimensional Lists

A **two-dimensional list** (2D list) is a list of lists, used to represent data in rows and columns—similar to a matrix or table.

Creating a 2D List:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

3.4 Tuples

A **tuple** is an **ordered, immutable** collection of items. Once created, its values **cannot be changed**. Tuples are used to store multiple items in a single variable, similar to lists, but with data integrity.

Creating a Tuple:

```
my_tuple = (10, 20, 30)
print(my_tuple) # Output: (10, 20, 30)
```

Tuple Operations:

- **Indexing:** `t[0]`
- **Slicing:** `t[1:3]`
- **Length:** `len(t)`
- **Concatenation:** `t1 + t2`
- **Membership:** `x in t`

3.5 Dictionaries

A **dictionary** is an **unordered, mutable** collection in Python that stores data in **key-value pairs**. Each key must be unique, and it allows fast lookup, insertion, and deletion.

Creating a Dictionary:

```
student = {
    "name": "Alice",
    "age": 21,
    "course": "Computer Science"
}
```

Retrieving Elements:

Retrieving data from a dictionary means accessing the value associated with a specific **key**. Python provides multiple ways to retrieve values safely and efficiently.

Using Square Brackets ([]):

```
student = {"name": "Alice", "age": 21}
print(student["name"]) # Output: Alice
```

Adding or Updating Elements:

In Python, **adding** a new key-value pair or **updating** an existing one uses the **same syntax**:

1. Using Assignment (dict[key] = value)

- **Add a new element** if the key doesn't exist.
- **Update the value** if the key already exists.

```
student = {"name": "Alice", "age": 21}
# Adding a new key-value pair
student["course"] = "Python"
# Updating an existing key's value
student["age"] = 22
print(student)
# Output: {'name': 'Alice', 'age': 22, 'course': 'Python'}
```

2. Using update() Method

- Accepts a dictionary or key-value pairs as arguments.

```
student.update({"grade": "A", "age": 23})
print(student)
# Output: {'name': 'Alice', 'age': 23, 'course': 'Python', 'grade': 'A'}
```

Removing Elements:

Python provides several methods to remove one or more elements from a dictionary safely and efficiently.

1. Using pop(key)

Removes the specified key and returns its value. Raises a KeyError if the key is not found.

```
student = {"name": "Alice", "age": 22, "course": "Python"}
student.pop("course")
print(student)
# Output: {'name': 'Alice', 'age': 22}
```

2. Using del Statement

Deletes a key-value pair directly.

```
del student["age"]
print(student)
# Output: {'name': 'Alice'}
```

3. Using clear()

Removes all items from the dictionary.

```
student.clear()
print(student)
```

4. Using Dictionary Comprehension (for conditional removal)

Example: Remove all keys with None values:

```
student = {"name": "Alice", "grade": None, "age": 22}
student = {k: v for k, v in student.items() if v is not None}
print(student)
# Output: {'name': 'Alice', 'age': 22}
```

3.6 Sets

A **set** is an unordered collection of unique elements. Sets are useful for membership tests, eliminating duplicates, and performing mathematical set operations like union, intersection, and difference.

Creating a Set:

```
my_set = {1, 2, 3}
print(my_set) # Output: {1, 2, 3}
# Using set() constructor
```

```
empty_set = set()
```

Adding Elements:

```
my_set.add(4)
print(my_set) # Output: {1, 2, 3, 4}
```

Removing Elements

- **remove()**: Removes item, raises error if not found
- **discard()**: Removes item, no error if not found
- **pop()**: Removes a random item
- **clear()**: Empties the set

```
my_set.remove(2)
my_set.discard(5) # No error even if 5 is not in the set
```

Set Operations:

Union: *Combines elements from both sets*

```
print(a | b) # {1, 2, 3, 4, 5}
```

Intersection: *Elements common to both sets*

```
print(a & b) # {3}
```

Difference: *Elements in a but not in b*

```
print(a - b) # {1, 2}
```

3.7 Strings in Python

A **string** is a sequence of characters enclosed in quotes (' ' or " "). Python provides powerful tools to **slice, test, search, and manipulate** strings.

1. String Slicing in Python

String slicing is used to extract a portion (substring) of a string using the syntax:

Parameters:

- **start** – index where the slice begins (inclusive)
- **stop** – index where the slice ends (exclusive)
- **step** – interval between characters (optional)

Examples:

```
text = "PythonProgramming"
```

Slice	Output	Explanation
text[0:6]	'Python'	Characters from index 0 to 5
text[6:]	'Programming'	From index 6 to end
text[:6]	'Python'	From start to index 5
text[-11:-1]	'Programmin'	Negative indexing (backwards)
text[::2]	'Pto rgamn'	Every 2nd character
text[::-1]	'gnimmargorPnohtyP'	Reversed string

2. String Testing Methods in Python

Python provides **built-in methods** to test the nature of string content. These methods return **Boolean values (True or False)** based on the content of the string.

Common String Testing Methods

Method	Description	Example Output
isalnum()	Returns True if all characters are alphanumeric (letters and digits)	'Python123'.isalnum() → True
isalpha()	Returns True if all characters are alphabetic	'Python'.isalpha() → True
isdigit()	Returns True if all characters are digits	'12345'.isdigit() → True
islower()	Returns True if all characters are lowercase	'python'.islower() → True
isupper()	Returns True if all characters are uppercase	'PYTHON'.isupper() → True
isspace()	Returns True if the string contains only whitespace	' '.isspace() → True
istitle()	Returns True if each word starts with an uppercase letter	'Python Is Fun'.istitle() → True

Examples:

```
s1 = "Python123"
s2 = "python"
s3 = "123"
s4 = " "
print(s1.isalnum()) # True
print(s2.isalpha()) # True
print(s3.isdigit()) # True
print(s4.isspace()) # True
```

3. String Searching

String searching methods help you **find the position** of a substring or **check its presence** within a string.

Common Searching Methods

Method	Description	Example Output
find(sub)	Returns the first index of sub, or -1 if not found	'hello'.find('l') → 2
rfind(sub)	Returns the last index of sub, or -1 if not found	'hello'.rfind('l') → 3
index(sub)	Like find(), but raises an error if sub not found	'hello'.index('e') → 1
rindex(sub)	Like rfind(), but raises error if not found	'hello'.rindex('l') → 3
'sub' in string	Returns True if sub is in the string	'he' in 'hello' → True
'sub' not in string	Returns True if sub is not in the string	'z' not in 'hello' → True

Examples:

```
text = "Python Programming"

print(text.find("gram")) # 10
print(text.rfind("m")) # 17
print(text.index("P")) # 0
print("Pro" in text) # True
print("Java" not in text) # True
```

4. String Manipulation

String manipulation involves changing or transforming strings using various built-in methods. These methods help clean, format, and prepare text data for processing.

common String Manipulation Methods

Method	Description	Example
lower()	Converts all characters to lowercase	'Python'.lower() → 'python'
upper()	Converts all characters to uppercase	'Python'.upper() → 'PYTHON'
capitalize()	Capitalizes the first character	'python'.capitalize() → 'Python'
title()	Capitalizes the first letter of each word	'python programming'.title() → 'Python Programming'

strip()	Removes leading and trailing whitespace	' hello '.strip() → 'hello'
replace(old, new)	Replaces all occurrences of old with new	'Python'.replace('P', 'J') → 'Jython'
split()	Splits a string into a list	'a,b,c'.split(',') → ['a', 'b', 'c']
join(list)	Joins elements of a list into a string	' '.join(['Python', 'is', 'fun']) → 'Python is fun'
startswith(sub)	Checks if string starts with sub	'hello'.startswith('he') → True
endswith(sub)	Checks if string ends with sub	'hello'.endswith('lo') → True

Examples:

```
text = " Python Programming "
```

```
print(text.lower())      # ' python programming '
print(text.strip())      # 'Python Programming'
print(text.replace("Python", "Java")) # ' Java Programming '
print(text.split())      # ['Python', 'Programming']
print("-".join(['Python', 'Rocks'])) # 'Python-Rocks'
```